```
import networkx as nx


def optimize_route():
  # 1. Initialize the City Road Network Graph
  G = nx.DiGraph()

  # 2. Define Intersections/Nodes
  # Origin: Church Street
  # Destination: Commercial Street
  # Intermediate points: MG Road, Kamaraj Road, Dickenson Road, Central
Street
  nodes = ["Church_St", "MG_Road", "Kamaraj_Rd", "Dickenson_Rd",
"Central_St", "Commercial_St"]
  G.add_nodes_from(nodes)

  # 3. Define Road Segments (Edges) with Initial Parameters
  # Format: (Source, Target, Baseline Distance in km, Traffic Factor [1.0
= Free, 3.0 = Jam])
  road_segments = [
      ("Church_St", "MG_Road", 0.4, 1.2),
      ("MG_Road", "Kamaraj_Rd", 0.6, 2.5),       # Heavy traffic junction
      ("Kamaraj_Rd", "Commercial_St", 0.9, 1.8),

      ("Church_St", "Central_St", 0.5, 1.5),
      ("Central_St", "Dickenson_Rd", 0.8, 1.1),  # Alternative route, less
traffic
      ("Dickenson_Rd", "Commercial_St", 1.3, 1.4)
  ]

  # 4. Define Optimization Metrics & Weights
  # Baseline average urban driving speed: 20 km/h (0.33 km per minute)
  avg_speed_km_min = 0.33

  for source, target, distance, traffic in road_segments:
    # Calculate pure baseline travel time (minutes)
    baseline_time = distance / avg_speed_km_min
```

```
    # Calculate real-world dynamic time under traffic conditions
    actual_travel_time = baseline_time * traffic

    # Introduce a "frustration/signal penalty" for notoriously congested
arterial junctions
    signal_penalty = 1.5 if traffic > 2.0 else 0.0

    # Combined Multi-Objective Cost Function (Time + Stress Factor)
    total_optimized_cost = actual_travel_time + signal_penalty

    G.add_edge(
        source,
        target,
        distance=distance,
        traffic=traffic,
        cost=total_optimized_cost,
        time=actual_travel_time
    )

  # 5. Execute Optimization Engine (Dijkstra's Shortest Path Algorithm)
  origin = "Church_St"
  destination = "Commercial_St"

  try:
    optimal_path = nx.dijkstra_path(G, source=origin, target=destination,
weight="cost")

    # Calculate summary metrics for the chosen optimal path
    total_dist = 0
    total_time = 0

    for i in range(len(optimal_path) - 1):
      edge_data = G[optimal_path[i]][optimal_path[i+1]]
      total_dist += edge_data["distance"]
      total_time += edge_data["time"]

    # 6. Format & Display Route Output
    print("--- Optimal Route Navigation Profile ---")
    print(f"Optimal Path: {' -> '.join(optimal_path)}")
    print(f"Total Distance: {total_dist:.2f} km")
```

```
    print(f"Estimated Travel Time: {total_time:.1f} minutes")

  except nx.NetworkXNoPath:
    print("Error: No navigable route found between origin and
destination.")


if __name__ == "__main__":
    optimize_route()
```

OUTPUT:

